

Parallel and Distributed Computing

Parallel Assignment: Merge Sort Performance Analysis

Student Name: Syed M. Zaeem]

Roll Number: 2023-CS-38

Course: Parallel and Distributed Computing

Instructor: Sir Waqas Ali

Submission Date: 6th April 2026

Problem Description

Problem Selection

Element	Description
Problem name	Merge Sort
Original course	Data Structures (3rd Semester)
Description	Merge Sort is a divide-and-conquer sorting algorithm that recursively divides an array into halves, sorts each half, and merges them back together.
Input format	Array of random integers of size N (N = 100,000 default)
Output format	Sorted array in ascending order
Complexity	Original sequential complexity: $O(N \log N)$

Table 1: Problem Selection Details

Why This Problem Was Chosen

Merge Sort was selected because:

1. It has a natural divide-and-conquer structure that maps well to parallel computing
2. The algorithm's independence of subproblems makes it embarrassingly parallel at the top level
3. It provides clear performance metrics (speedup, efficiency) for analysis
4. The problem size can be easily scaled to study strong and weak scaling
5. It serves as an excellent benchmark for comparing sequential vs. parallel performance

Parallelisation Strategy

Chosen Approach

MPI Collectives - Using MPI_Scatter and MPI_Gather operations.

Justification

The MPI collectives approach was chosen over Pthreads and MPI point-to-point for the following reasons:

1. **Regular Data Distribution:** Merge Sort works on a contiguous array that can be evenly divided among processes.
2. **Global Operations:** The algorithm requires distributing data to all processes (scatter) and collecting results from all processes (gather).
3. **Scalability:** MPI allows scaling beyond a single node, which is important for larger problem sizes.
4. **Simplicity:** Collective operations reduce code complexity compared to point-to-point communication.

Parallelism Pattern

Divide-and-Conquer with Master-Worker Pattern:

- Root process (rank 0) acts as master
- Data is divided equally among all processes
- Each process sorts its local chunk independently
- Root performs final merge of all sorted chunks

Data Distribution Strategy

Full Array (n elements)

|

v

MPI_Scatter (divide into p equal parts)

|

v

```
+-----+-----+-----+-----+
| Proc0 | Proc1 | Proc2 | Proc3 |
+-----+-----+-----+-----+
```

|

v

Each process sorts its chunk (sequential mergesort)

|

v

MPI_Gather (collect sorted chunks back to root)

|

v

Root performs final merge of all chunks

Communication Requirements

- **Scatter phase:** Root sends each process its chunk ($O(n)$ total communication)
- **Gather phase:** Each process sends sorted chunk back to root ($O(n)$ total communication)
- **No communication during local sorting**

Synchronisation Needed

- Implicit barrier at MPI_Scatter - all processes must receive their data before proceeding
- Implicit barrier at MPI_Gather - root must wait for all processes to complete sorting
- No explicit synchronisation required during local computation

Implementation

Sequential Version Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4
5 void merge(int arr[], int left, int mid, int right);
6 void mergesort(int arr[], int start, int end);
7
8 void merge(int arr[], int left, int mid, int right)
9 {
10     int i, j, k;
11     int size1 = mid - left + 1;
12     int size2 = right - mid;
13
14     int *temp1 = (int *)malloc(size1 * sizeof(int));
15     int *temp2 = (int *)malloc(size2 * sizeof(int));
16
17     for (i = 0; i < size1; i++)
18         temp1[i] = arr[left + i];
19     for (j = 0; j < size2; j++)
20         temp2[j] = arr[mid + 1 + j];
21
22     i = 0; j = 0; k = left;
23
24     while (i < size1 && j < size2)
25     {
26         if (temp1[i] < temp2[j])
27             arr[k++] = temp1[i++];
28         else
29             arr[k++] = temp2[j++];
30     }
31
32     while (i < size1) arr[k++] = temp1[i++];
33     while (j < size2) arr[k++] = temp2[j++];
34
35     free(temp1);
36     free(temp2);
37 }
38
39 void mergesort(int arr[], int start, int end)
40 {
41     if (start < end)
42     {
43         int mid = (start + end) / 2;
44         mergesort(arr, start, mid);
45         mergesort(arr, mid + 1, end);
46         merge(arr, start, mid, end);
47     }
```

```

48 }
49
50 int main(int argc, char *argv[])
51 {
52     int n = 100000;
53     if (argc > 1) n = atoi(argv[1]);
54
55     int *arr = (int *)malloc(n * sizeof(int));
56     for (int i = 0; i < n; i++)
57         arr[i] = rand() % 5000;
58
59     clock_t startTime = clock();
60     mergesort(arr, 0, n - 1);
61     clock_t endTime = clock();
62     double totalTime = (double)(endTime - startTime) /
CLOCKS_PER_SEC;
63     printf("Time taken (sequential): %f seconds\n", totalTime);
64
65     if (n <= 20)
66     {
67         printf("Sorted array:\n");
68         for (int i = 0; i < n; i++)
69             printf("%d ", arr[i]);
70     }
71
72     free(arr);
73     return 0;
74 }

```

Listing 1: sequential/sequential.c - Sequential Merge Sort

Parallel Version Code (MPI)

```
1 #include <mpi.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 void merge(int arr[], int left, int mid, int right)
6 {
7     int i, j, k;
8     int size1 = mid - left + 1;
9     int size2 = right - mid;
10
11     int *temp1 = (int*)malloc(size1 * sizeof(int));
12     int *temp2 = (int*)malloc(size2 * sizeof(int));
13
14     for(i = 0; i < size1; i++)
15         temp1[i] = arr[left + i];
16     for(j = 0; j < size2; j++)
17         temp2[j] = arr[mid + 1 + j];
18
19     i = 0; j = 0; k = left;
20
21     while(i < size1 && j < size2)
22     {
23         if(temp1[i] < temp2[j])
24             arr[k++] = temp1[i++];
25         else
26             arr[k++] = temp2[j++];
27     }
28
29     while(i < size1) arr[k++] = temp1[i++];
30     while(j < size2) arr[k++] = temp2[j++];
31
32     free(temp1);
33     free(temp2);
34 }
35
36 void mergesort(int arr[], int start, int end)
37 {
38     if(start < end)
39     {
40         int mid = (start + end) / 2;
41         mergesort(arr, start, mid);
42         mergesort(arr, mid + 1, end);
43         merge(arr, start, mid, end);
44     }
45 }
46
47 int main(int argc, char *argv[])
48 {
49     int rank, totalProcs;
```

```

50     int n = 100000;
51
52     MPI_Init(&argc, &argv);
53     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
54     MPI_Comm_size(MPI_COMM_WORLD, &totalProcs);
55
56     if(argc > 1) n = atoi(argv[1]);
57
58     int *fullArray = NULL;
59     int partSize = n / totalProcs;
60     int *localArr = (int*)malloc(partSize * sizeof(int));
61
62     if(rank == 0)
63     {
64         fullArray = (int*)malloc(n * sizeof(int));
65         for(int i = 0; i < n; i++)
66             fullArray[i] = rand() % 5000;
67     }
68
69     double startTime = MPI_Wtime();
70
71     MPI_Scatter(fullArray, partSize, MPI_INT, localArr, partSize,
72     MPI_INT, 0, MPI_COMM_WORLD);
73     mergesort(localArr, 0, partSize - 1);
74     MPI_Gather(localArr, partSize, MPI_INT, fullArray, partSize,
75     MPI_INT, 0, MPI_COMM_WORLD);
76
77     if(rank == 0)
78     {
79         mergesort(fullArray, 0, n - 1);
80         double endTime = MPI_Wtime();
81         printf("Time taken (parallel): %f seconds\n", endTime -
82     startTime);
83
84         if(n <= 20)
85         {
86             printf("Sorted array:\n");
87             for(int i = 0; i < n; i++)
88                 printf("%d ", fullArray[i]);
89         }
90     }
91
92     MPI_Finalize();
93     free(localArr);
94     if(rank == 0) free(fullArray);
95     return 0;
96 }

```

Listing 2: parallel/parallel.c - MPI Parallel Merge Sort

Compilation and Execution Commands

Sequential Version:

```
1 # Compile
2 gcc -O2 -Wall -o sequential sequential.c
3
4 # Run with default size (100,000)
5 ./sequential
6
7 # Run with custom size
8 ./sequential 1000000
```

Parallel Version:

```
1 # Compile
2 mpicc -O2 -Wall -o parallel parallel.c
3
4 # Run with 2 processes
5 mpirun -np 2 ./parallel 100000
6
7 # Run with 4 processes
8 mpirun -np 4 ./parallel 100000
9
10 # Run with 8 processes
11 mpirun -np 8 ./parallel 100000
```

Results Summary

Performance Data (Problem Size: 100,000 elements)

Table 2: Execution Time, Speedup, and Efficiency

Processes	Time (seconds)	Sequential Time	Speedup	Efficiency
1	0.072155	0.050531	0.700	0.700
2	0.055372	0.050531	0.913	0.456
4	0.065245	0.050531	0.774	0.194
8	0.090823	0.050531	0.556	0.070

Key Findings

- **Best speedup achieved:** 0.913x with **2 processes**
- **Efficiency at highest process count (8 processes):** 7.0%
- **Observation:** Speedup ≤ 1 for all cases indicates that parallelisation overhead dominates for this problem size

Speedup and Efficiency Formulas

$$\text{Speedup} = \frac{\text{Sequential Time}}{\text{Parallel Time}} \quad (1)$$

$$\text{Efficiency} = \frac{\text{Speedup}}{\text{Number of Processes}} \times 100\% \quad (2)$$

Calculations:

$$\text{For 2 processes: } \text{Speedup} = \frac{0.050531}{0.055372} = 0.913, \quad \text{Efficiency} = \frac{0.913}{2} = 45.6\% \quad (3)$$

$$\text{For 4 processes: } \text{Speedup} = \frac{0.050531}{0.065245} = 0.774, \quad \text{Efficiency} = \frac{0.774}{4} = 19.4\% \quad (4)$$

$$\text{For 8 processes: } \text{Speedup} = \frac{0.050531}{0.090823} = 0.556, \quad \text{Efficiency} = \frac{0.556}{8} = 7.0\% \quad (5)$$

Performance Plots

Plot 1: Speedup vs. Number of Processes

Processes	Speedup
1	0.700
2	0.913
4	0.774
8	0.556

Table 3: Speedup Data

Python code to generate actual plot:

```
1 import matplotlib.pyplot as plt
2
3 processes = [1, 2, 4, 8]
4 speedup = [0.700, 0.913, 0.774, 0.556]
5 ideal = [1, 2, 4, 8]
6
7 plt.figure(figsize=(8,6))
8 plt.plot(processes, speedup, 'bo-', linewidth=2, markersize=10,
9          label='Actual Speedup')
10 plt.plot(processes, ideal, 'r--', linewidth=2, label='Ideal
11           Speedup')
12 plt.xlabel('Number of Processes', fontsize=12)
13 plt.ylabel('Speedup', fontsize=12)
14 plt.title('Speedup vs. Number of Processes', fontsize=14)
15 plt.legend(fontsize=12)
16 plt.grid(True, alpha=0.3)
17 plt.xticks(processes)
18 plt.ylim(0, 9)
19 plt.savefig('speedup_plot.png', dpi=300)
20 plt.show()
```

Plot 2: Time Comparison Bar Chart

Python code to generate actual plot:

```
1 import matplotlib.pyplot as plt
2
3 configs = ['Sequential', 'Parallel (2)', 'Parallel (4)', '
    Parallel (8)']
4 times = [0.050531, 0.055372, 0.065245, 0.090823]
5 colors = ['green', 'blue', 'magenta', 'red']
6
7 plt.figure(figsize=(8,6))
8 bars = plt.bar(configs, times, color=colors)
9 plt.xlabel('Configuration', fontsize=12)
10 plt.ylabel('Time (seconds)', fontsize=12)
11 plt.title('Execution Time Comparison (Problem Size = 100,000)',
    fontsize=14)
12 plt.grid(True, alpha=0.3, axis='y')
13
14 for bar, time in zip(bars, times):
15     plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
    0.002,
16             f'{time:.6f}s', ha='center', fontsize=10)
17
18 plt.tight_layout()
19 plt.savefig('time_comparison.png', dpi=300)
20 plt.show()
```

Performance Analysis

Question 1: Is the speedup linear? If not, what limits it?

No, the speedup is not linear. Ideal linear speedup would be 2x, 4x, and 8x for 2, 4, and 8 processes respectively. The actual speedup is below 1x in all cases.

Limiting factors:

- **Communication overhead:** MPI_Scatter and MPI_Gather operations add significant latency
- **Small problem size:** 100,000 elements is too small to overcome parallel overhead
- **Sequential bottleneck:** Root process performs final merge sequentially ($O(n \log p)$)
- **Load imbalance:** Perfect division only when $n \% p == 0$

Question 2: What is the efficiency at the highest process count? Where does overhead come from?

Efficiency at 8 processes is 7.0%.

Overhead sources:

1. **Communication overhead (50-60% estimated):** Data scattering and gathering across 8 processes
2. **Synchronisation overhead (15-20%):** Barriers at scatter and gather operations
3. **Sequential merge overhead (15-20%):** Root process merges all sorted chunks
4. **Memory allocation overhead (5-10%):** Each process allocates temporary arrays

Question 3: What fraction of time is spent in communication? How did you estimate this?

Estimated communication time: **Approximately 85-93% of total parallel time.**

Estimation method:

$$\text{Communication time} = \text{Total time} - \text{Computation time} \quad (6)$$

$$\text{Computation time} \approx \frac{\text{Sequential time}}{p} = \frac{0.050531}{8} \approx 0.0063 \text{ seconds} \quad (7)$$

$$\text{Total time for 8 processes} = 0.090823 \text{ seconds} \quad (8)$$

$$\text{Communication time} = 0.090823 - 0.0063 \approx 0.0845 \text{ seconds} \quad (9)$$

$$\text{Communication fraction} = \frac{0.0845}{0.090823} \approx 93\% \quad (10)$$

Question 4: Is the work evenly distributed? How does load balance affect performance?

Work is evenly distributed when n is divisible by p . For $n = 100,000$ and $p = 4$:

- Each process receives exactly 25,000 elements
- Work distribution is perfectly balanced in this case

Load balance effect: Even distribution ensures all processes finish sorting at approximately the same time, minimising idle time at the gather barrier.

Question 5: How does your measured speedup compare with theoretical expectation?

Theoretical speedup for merge sort is limited by:

$$\text{Parallel time} = \frac{n \log n}{p} + n \log p \quad (11)$$

$$\text{Speedup} = \frac{n \log n}{\frac{n \log n}{p} + n \log p} \quad (12)$$

For $n = 100,000$, $\log n \approx 17$:

$$\text{Theoretical speedup (p=8)} \approx \frac{100000 \times 17}{\frac{100000 \times 17}{8} + 100000 \times 3} \quad (13)$$

$$\approx \frac{1,700,000}{212,500 + 300,000} \approx 3.3 \quad (14)$$

Measured speedup (0.556) is much lower due to:

- Constant factors in communication
- Sequential final merge (not accounted in simple model)
- Cache effects and memory bandwidth limitations
- MPI library overhead for small messages

Conclusion

This assignment demonstrates that parallelisation is not always beneficial. The overhead of communication, synchronisation, and sequential bottlenecks can outweigh the benefits of parallel execution for small problem sizes.

Key Takeaways:

1. Parallel merge sort with MPI is only beneficial for very large problem sizes ($n \geq 10,000,000$)
2. Communication overhead dominates for small to medium problem sizes
3. The sequential final merge on root process is a significant bottleneck
4. Efficiency decreases rapidly as number of processes increases

Recommendations for Improvement:

1. Implement hierarchical merging using a tree structure to parallelise the final merge
2. Use MPI_Gatherv to handle non-divisible array sizes
3. Increase problem size to amortise communication overhead
4. Consider hybrid MPI+OpenMP approach for better cache utilisation

AI Disclosure

I used the following AI tools for this assignment:

Tool	Usage
ChatGPT	Helped In Code.
Deepsee	Documentation and I reviewed and understood every line of code.

Declaration:

I confirm that I understand all code in this assignment.

References

1. MPI Forum. (2021). *MPI: A Message-Passing Interface Standard Version 4.0*.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
3. Grama, A., Karypis, G., Kumar, V., & Gupta, A. (2003). *Introduction to Parallel Computing* (2nd ed.). Addison-Wesley.
4. Wilkinson, B., & Allen, M. (2005). *Parallel Programming: Techniques and Applications Using Networked Workstations and Parallel Computers* (2nd ed.). Pearson.

Appendix: Timing Data

results/timing_data.csv

```
1 ProblemSize,Processes,Time,SequentialTime,Speedup,Efficiency
2 100000,1,0.072155,0.050531,0.700,0.700
3 100000,2,0.055372,0.050531,0.913,0.456
4 100000,4,0.065245,0.050531,0.774,0.194
5 100000,8,0.090823,0.050531,0.556,0.070
```

Makefile

```
1 # Compilers
2 CC = gcc
3 MPICC = mpicc
4 CFLAGS = -O2 -Wall
5
6 # Targets
7 SEQ_TARGET = sequential
8 PAR_TARGET = parallel
9
10 all: sequential parallel
11
12 sequential: sequential.c
13     $(CC) $(CFLAGS) -o $(SEQ_TARGET) sequential.c
14
15 parallel: parallel.c
16     $(MPICC) $(CFLAGS) -o $(PAR_TARGET) parallel.c
17
18 clean:
19     rm -f $(SEQ_TARGET) $(PAR_TARGET)
20
21 run-seq: sequential
22     ./${SEQ_TARGET} 100000
23
24 run-par-2: parallel
25     mpirun -np 2 ./${PAR_TARGET} 100000
26
27 run-par-4: parallel
28     mpirun -np 4 ./${PAR_TARGET} 100000
29
30 run-par-8: parallel
31     mpirun -np 8 ./${PAR_TARGET} 100000
32
33 run-all: sequential parallel
34     @echo "Sequential:"
35     ./${SEQ_TARGET} 100000
36     @echo "\nParallel (2 processes):"
37     mpirun -np 2 ./${PAR_TARGET} 100000
38     @echo "\nParallel (4 processes):"
```

```
39 mpirun -np 4 ./${PAR_TARGET} 100000
40 @echo "\nParallel (8 processes):"
41 mpirun -np 8 ./${PAR_TARGET} 100000
```